INTEGER ARITHMETIC & OVERFLOW

Addition/Subtraction Hardware: Subtraction ($A-B$) is performed by adding the 2's complement of B to A ($A+ \sim B+1$). The same ALU does both.

Overflow Detection

Unsigned Overflow: Occurs if there is a Carry-Out from the Most Significant Bit (MSB).

Signed Overflow (2's Complement):

- Occurs if adding two positive numbers yields a negative result.
- Occurs if adding two negative numbers yields a positive result.
- **Hardware Check:** $V = C_{in} \oplus C_{out}$ (Overflow happens if the Carry-In to the MSB is different from the Carry-Out of the MSB).

Note: Adding a positive and a negative number can **NEVER** result in an overflow.

MULTIPLICATION HARDWARE

1. Standard/Unoptimized Multiplier:

- 64-bit ALU, 64-bit Product Register, 64-bit Multiplicand Register.
- Multiplicand starts in the right half. Multiplier in a separate register.
- **Algorithm:** If LSB of Multiplier = 1, add Multiplicand to Product. Shift Multiplicand **Left**. Shift Multiplier **Right**.

DIVISION HARDWARE

Restoring Division Algorithm:

1. Shift Remainder/Quotient register left 1 bit.
2. Subtract Divisor from the left half (Remainder).
3. **If result $\geq$ 0:** Shift a `1` into the Quotient.
4. **If result $<$ 0:** Shift a `0` into the Quotient AND **Restore** (Add Divisor back to the Remainder).

Divide by zero: Hardware ignores it. Software must check.

2. Optimized Multiplier (Saves Hardware):

- 32-bit ALU, 64-bit Product Register, 32-bit Multiplicand.
- **Setup:** Multiplicand stays fixed. Multiplier is placed in the **LOWER half** of the Product register. Upper half is 0.

Decimal to IEEE 754 Conversion

Example: Convert -21.75 to Double Precision.

1. **Determine Sign:** Negative, so $S = 1$.
2. **Convert to Binary:** $21 = 10101_2$. $0.75 = 1/2 + 1/4 = .11_2$. Binary magnitude = 10101.11_2.
3. **Normalize:** Shift binary point left until 1.XXXX. $10101.11_2 \rightarrow 1.010111 \times 2^4$.
4. **Determine Fraction:** Drop the leading "1.". Fraction = `010111` (pad right with 0s to fill 52 bits).
5. **Calculate Stored Exponent:** Stored Exp = Actual Exp + Bias. Stored Exp = 4 + 1023 = 1027. Convert 1027 to 11-bit binary: `10000000011`.
6. **Assemble:** `[S] [Exponent] [Fraction]`
   `1 10000000011 01011100...000`

IEEE 754 FLOATING-POINT FORMAT

Formula: $X = (-1)^S \times (1.\text{Fraction}) \times 2^{(\text{Exponent}-\text{Bias})}$

- **Field, Single Precision, Double Precision**
- **Total Bits:** 32 64
- **Sign (S):** 1 bit 1 bit
- **Exponent (E):** 8 bits 11 bits
- **Fraction (F):** 23 bits 52 bits
- **Bias: 127 1023**

FLOATING POINT SPECIAL VALUES

Hardware relies on the **Exponent** to flag special cases:

- **Zero:** Exp = `00...0`, Frac = `00...0`.
- **Denormalized:** Exp = `00...0`, Frac $\neq 0$.
  - Used for gradual underflow (extremely small numbers).
  - **NO hidden 1!** Formula: $(-1)^S \times (0.\text{Frac}) \times 2^{1-\text{Bias}}$
- **Infinity ($\pm\infty$):** Exp = `11...1`, Frac = `00...0`. (Used for divide by zero).
- **NaN (Not a Number):** Exp = `11...1`, Frac $\neq 0$. (Used for 0/0 or $\sqrt{-1}$).

SIMD / SUBWORD PARALLELISM

- **Data-Level Parallelism:** Operating on short vectors simultaneously (e.g., 4 pixels at once).
- **ARM NEON:** Uses 32 128-bit registers (`V0-V31`).
- *Example:* A 128-bit adder can do sixteen 8-bit adds, eight 16-bit adds, or four 32-bit adds in parallel.

FLOATING-POINT MULTIPLICATION

1. **Add Exponents:** *Careful!* If you just add the two stored exponents, you add the bias twice. New Exp = (Stored Exp_1 + Stored Exp_2) − Bias
2. **Multiply Significands:** Multiply the 1.XX values like normal binary multiplication.
3. **Normalize:** Shift the product to get 1.XXXX and adjust the new exponent accordingly.
4. **Sign:** XOR the original sign bits. (Pos × Neg = Neg).

FP INSTRUCTIONS (LEGV8)

- Uses separate registers: `S0-S31` (32b) & `D0-D31` (64b). `Sn` is the lower half of `Dn`.
- Math: `FADDS`, `FSUBS`, `FMULS`, `FDIVS` (Single). Change trailing 'S' to 'D' for Double.
- Loads/Stores: `LDURS`, `STURS`, `LDURD`, `STURD`.
- Compare: `FCMPS`, `FCMPD`. (Sets FP condition flags).

Step-by-Step FP Addition

1. Align Binary Points:

- Compare Exponents. Find the number with the **smaller** exponent.
- Shift the significand (fraction with hidden 1 added back) of the smaller number to the **RIGHT** by the difference between the exponents.
- *Example:* If $Exp1 = 5$ and $Exp2 = 3$, shift Significand 2 right by 2 bits. Now both exponents are 5.

2. Add Significands:

- Perform standard binary addition on the aligned significands. (Remember to include the hidden 1s!).

3. Normalize Result & Check Over/Underflow:

- Shift the resulting significand left or right to get it back into normalized 1.XXXX form.
- **If you shift right** (e.g., 10.101 → 1.0101), you must **ADD 1** to the exponent for each shift.
- **If you shift left** (e.g., 0.011 → 1.1), you must **SUBTRACT 1** from the exponent for each shift.
- Check if the new Exponent exceeds the max (overflow) or min (underflow) bounds.

4. Round and Renormalize:

- If the significand has more bits than the Fraction field allows (23 for single, 52 for double), round it according to IEEE rules.
- If rounding causes a carry that breaks normalization (e.g., 1.111 + .001 = 10.000), renormalize again (shift right 1, add 1 to Exp).

Optimized Multiply Trace Table Rules

Columns needed: Iteration, Step, Multiplicand, Product.

Product Format: `[Carry] Upper_Half | Lower_Half`

1. Check the LSB (rightmost bit) of the Product register.
2. **If LSB == 1:** Add the Multiplicand to the **Upper Half** of the Product register. Save any Carry-Out in a temporary Carry bit `[C]`.
3. **If LSB == 0:** Do nothing (skip addition).
4. **Shift:** Shift the *entire* Product register (including the Carry bit) **Right by 1 bit**. The Carry bit drops into the MSB of the Upper Half.
5. Repeat N times (where N is the number of bits).

SIGN-EXTEND UNIT USAGE

Which instructions use the 64b output from Sign-Extend?

- **LDUR / STUR:** Uses 9-bit constant offset. Sign-extended to 64b, routed via ALUSrc mux to ALU to add to base register.
- **CBZ / B:** Uses 19-bit or 26-bit instruction offset. Sign-extended to 64b, then shifted left by 2 and sent to the Branch Adder (PC + offset).
- **NOT used by:** R-type (ADD, SUB, AND, ORR). These only use register values. ALUSrc routes "Read Data 2" instead.

LOGIC DESIGN BASICS

Combinational Logic: Outputs depend ONLY on current inputs. No memory/state.

- **ALU:** Performs arithmetic and logical ops.
- **Adders:** Two used (PC + 4, and Branch Target).
- **Multiplexors (Mux):** Routes data based on a control signal (0 or 1).
- **Control Units:** Main Control and ALU Control.
- **Sign-Extend:** Expands 9, 19, or 26-bit offsets to 64b.
- **Shift Left 2:** Multiplies branch offset by 4.
- **Logic Gates:** AND / OR gates for branch decisions.

Sequential Logic: State elements. Store data and update synchronously with a clock edge.

- **PC (Program Counter):** Holds current memory address.
- **Instruction Memory:** Stores the program's code.
- **Registers:** The 32 general-purpose 64-bit registers.
- **Data Memory:** Read/written during LDUR/STUR.

INSTRUCTION EXECUTION CYCLE

1. Fetch: PC → IM. Update PC = PC + 4.
2. Decode: Read RF. Control unit sets muxes.
3. Execute: ALU performs math or calculates addresses.
4. Memory: Access Data Memory (if LDUR/STUR).
5. Write-Back: Write result back to RF.

UNOPTIMIZED MULTIPLICATION TABLE

Iteration — Multiplier — Multiplicand — Product

OPTIMIZED MULTIPLICATION TABLE

Iteration — Step Desc — Multiplicand — Product

A x B — Multiplicand x Multiplier

DIVISION TABLE

Quotient — Divisor — Remainder
A/B — Divisor = A 0000 — Remainder = 0000 B — Quotient = 0000

DATAPATH CONTROL SIGNALS

The Main Control Unit decodes the instruction and sets the Muxes and read/write flags.

Control Signal Master Table

R2=Reg2Loc, UB=UncondBr, Br=Branch, M=MemR/W, M2=MemtoReg, AOp=ALUOp, AS=ALUSrc, RW=RegWrite, AC=ALUControl

Instr	R2	UB	Br	M	M2	AOp	AS	RW	AC
ADD	0	0	0	00	0	10	0	1	0010
SUB	0	0	0	00	0	10	0	1	0110
LDUR	X	0	0	10	1	00	1	1	0010
STUR	1	0	0	01	X	00	1	0	0010
CBZ	1	0	1	00	X	01	0	0	0111
B	X	1	X	00	X	XX	X	0	XXXX

Justifying "Don't Cares" (X):

- **LDUR (R2=X):** Does not read a second register; only reads the base register and uses immediate offset.
- **STUR & CBZ (M2=X):** RegWrite is 0. Since no data is saved to the register file, it doesn't matter what data the MemtoReg mux routes to it.
- **B (Unconditional):** Uncondbranch=1 forces the PC mux to jump. Therefore, it doesn't matter what the ALU calculates (AOp, AS, AC), what registers are read (R2), or what the CBZ AND-gate outputs (Br).

Branch Target Math (Shift Left 2)

Why Shift Left 2? The offset stored in a branch instruction is the number of *instructions*. Memory is byte-addressed (1 instruction = 4 bytes). Shifting a binary number left by 2 multiplies it by 4, converting the instruction offset to a byte offset.

Step-by-Step Calculation (CBZ / B):

1. **Calculate Offset:** Target Line - Current Line.
2. **Determine Binary:** Convert offset to binary. If negative, use **2's Complement** (Invert bits, Add 1).
3. **Sign-Extend:** Pad the left side with 1s (if negative) or 0s (if positive) until it is exactly 64 bits long.
4. **Shift Left 2:** Shift the entire 64-bit string left by 2 bits. (Append `00` to the right side, drop the two leftmost bits to maintain exactly 64 bits).

Example (Jump back 5 lines): Offset = -5. Binary +5 = `...00101`. Invert = `...11010`. Add 1 = `...11011`. Sign Extended = `1111...1111 1011`. Shift Left 2 = `1111...1110 1100` (Decimal -20).

Signal Definitions:

- **Reg2Loc:** `0` reads Rm (R-types). `1` reads Rt (STUR, CBZ).
- **Uncondbranch:** `1` for B. `0` for others.
- **Branch:** `1` for CBZ. `0` for others.
- **MemR/W (Combined):** `00` (Hold), `10` (Read/LDUR), `01` (Write/STUR).
- **MemtoReg:** `1` (Data Memory → Reg). `0` (ALU → Reg).
- **ALUOp:** `10` (R-type), `00` (Load/Store), `01` (CBZ).
- **ALUSrc:** `0` (Reg 2 → ALU). `1` (Sign-Ext Offset → ALU).
- **RegWrite:** `1` (Writing to a register). `0` (Not writing).
- **ALU Control (4b):** `0010` (Add), `0110` (Sub), `0111` (Pass input B).

EXTENDED MULTIPLICATION (128-BIT)

Multiplying two 64-bit registers produces a 128-bit result, which cannot fit in a single register. The ISA provides instructions to capture both halves:

- `MUL X3, X1, X2`: Calculates and stores the **Lower 64 bits** of the product.
- `SMULH X4, X1, X2`: Calculates and stores the **Upper 64 bits** (Signed).
- `UMULH X4, X1, X2`: Calculates and stores the **Upper 64 bits** (Unsigned).

Critical Path & Clock Speed

In a Single-Cycle Datapath, the clock cycle must be long enough for the slowest instruction to complete. **The longest delay is ALWAYS the Load (`LDUR`) instruction.**

- **Path:** Instr Mem → Reg File (Read) → ALU → Data Mem (Read) → Reg File (Write).
- **Impact:** The physical time it takes for a Load to travel this entire path dictates the absolute minimum clock cycle time (T_c) and the maximum clock frequency (f) for the entire CPU.

ALU CONTROL (2-LEVEL DECODING)

The Main Control Unit doesn't micromanage the ALU. It sends a 2-bit **ALUOp** signal to a smaller "ALU Control Unit", which then outputs the final 4-bit command.

- `ALUOp = 00` **(Memory):** Used for `LDUR` & `STUR`. The ALU Control immediately outputs `0010` (Add) to calculate the address.
- `ALUOp = 01` **(Branch):** Used for `CBZ`. The ALU Control outputs `0111` (Pass Input B) to check if the register is zero.
- `ALUOp = 10` **(R-Type):** Used for `ADD`, `SUB`, `AND`, `ORR`. The ALU Control looks at the 11-bit **Opcode** of the instruction to decide the final 4-bit command (`0010`, `0110`, `0000`, etc.).

DATAPATH USAGE: WHO USES WHAT?

Every instruction must first do **Instruction Fetch**: PC → Instr Mem, and PC+4 Adder → PC. After fetch, the paths diverge:

1. R-Format (`ADD, SUB, AND, ORR`)

- **Active:** Reg File (Reads Rm & Rn, Writes Rd), ALU.
- **Bypassed/Ignored:** Sign-Extend, Data Memory, Branch Adders.
- Instr Mem + Reg Read + ALU + Reg Write

2. Load (`LDUR`)

- **Active:** Reg File (Reads Rn, Writes Rt), Sign-Extend (9b offset), ALU (Adds Base+Offset), Data Memory (Reads).
- **Bypassed/Ignored:** Reg File "Read 2" port, Branch Adders.
- Instr Mem + Reg Read + ALU + Data Mem + Reg Write (CRITICAL PATH)

3. Store (`STUR`)

- **Active:** Reg File (Reads Rn & Rt), Sign-Extend (9b offset), ALU (Adds Base+Offset), Data Memory (Writes).
- **Bypassed/Ignored:** Reg File "Write" port (does not update registers), Branch Adders.
- Instr Mem + Reg Read + ALU + Data Mem

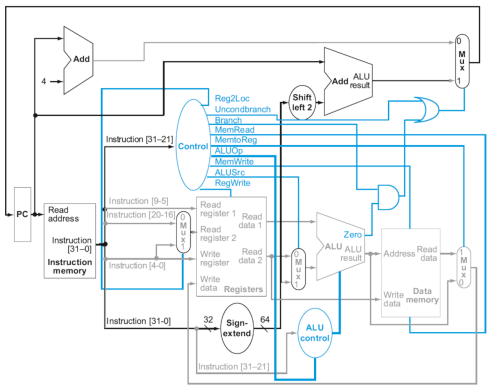
4. Conditional Branch (`CBZ`)

- **Active:** Reg File (Reads Rt via Reg2Loc), ALU (Passes B, outputs Zero flag), Sign-Extend (19b offset), Shift Left 2, Top Branch Adder, AND Gate.
- **Bypassed/Ignored:** Reg File "Write" port, Data Memory.
- Instr Mem + Reg Read + ALU

5. Unconditional Branch (`B`)

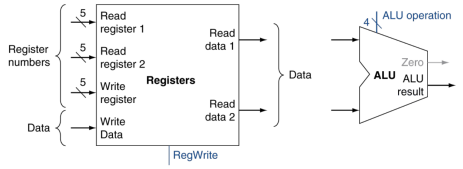
- **Active:** Sign-Extend (26b offset), Shift Left 2, Top Branch Adder, OR Gate (forces PC Mux).
- **Bypassed/Ignored (Everything else!):** Register File, ALU, Data Memory.
- Instr Mem + Sign Extend + Branch Adder

Datapath With B Added



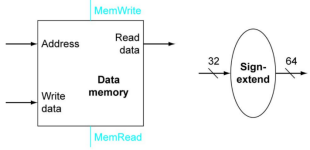
R-Format Instructions

- Read two register operands
- Perform arithmetic/logical operation
- Write register result

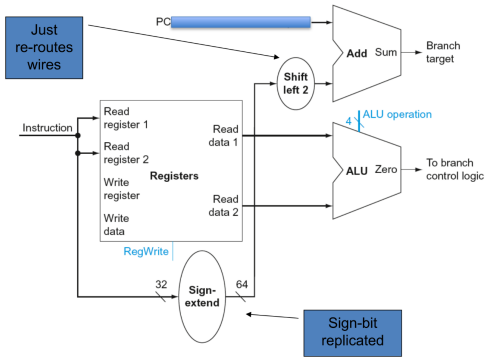


Load/Store Instructions

- Read register operands
- Calculate address using 9-bit offset
  - Use ALU, but sign-extend offset
  - Sign-extension unit takes 32-bit instruction. Then, selects 9b offset for load/store (19b offset for CBZ).
- Load: Read memory and update register
- Store: Write register value to memory



Branch Instructions



R-Type/Load/Store Datapath

